

Requerimiento de desarrollo — Divagando (generador de citas en Rust)

Contexto

Divagando es una app de escritorio para **aprender Rust**. Muestra citas célebres guardadas en SQLite. La interfaz es GTK4/Libadwaita escrita en Blueprint (ver `AGENTS.md`).

Estado actual (commit 438b991):

- `src/config/db.rs` tiene `get_engine()`, una conexión única guardada con `OnceLock<Mutex<SqliteConnection>>`.
- `Cargo.toml` tiene `diesel 2.3 (sqlite)`, `dotenvy` y `libsqlite3-sys (bundled)`.
- No existen `migrations/`, `models.rs`, `schema.rs` ni `ui/`, y `docs/` está vacío.

Entorno detectado:

- Windows 10 con toolchain `stable-x86_64-pc-windows-msvc` (rustc 1.98).
- No están instalados `diesel CLI`, `blueprint-compiler` ni `GTK`.

Regla de mentor (`AGENTS.md`): el usuario escribe el código Rust. Cada tarea indica **qué construir**, **qué conceptos de Rust aparecen** y **qué partes son verbosas y conviene explicar** cuando se lleguen a ellas.

Tarea 0 — Preparación del entorno (bloquea las tareas 1, 3 y 5)

1. Instalar la CLI de Diesel solo con soporte SQLite: `cargo install diesel_cli --no-default-features --features sqlite-bundled`.
2. Instalar GTK4 y Libadwaita para MSVC con **gvsbuild**: `gvsbuild build gtk4 libadwaita`. Agregar al PATH y a `PKG_CONFIG_PATH` las rutas `C:\gtk-build\gtk\x64\release\bin` y `...\lib\pkgconfig`. Es la vía que recomienda el libro de gtk-rs para MSVC.
3. Instalar `blueprint-compiler`, que es una herramienta de Python (desde su repositorio con Python 3).
4. Verificar: `diesel --version`, `pkg-config --modversion gtk4` y `blueprint-compiler --version`.

Aviso: en Windows este es el paso con más fricción de todo el proyecto. Es mejor resolverlo al inicio.

Tarea 1 — Base de datos, esquema y versionamiento (equivalente a Alembic)

Objetivo: que la estructura de la BD esté definida en Rust y versionada con migraciones reversibles.

Pasos

1. Ejecutar `diesel setup`. Crea `diesel.toml` y la carpeta `migrations/`. Diesel lee `DATABASE_URL`, así que hay que añadirla al `.env` o unificarla con `BD_NAME`.
2. **Definir el esquema en Rust:** escribir a mano `src/schema.rs` con la macro `diesel::table!`. Tabla `citas` con los campos:
 - `id` (entero, PK autoincremental)
 - `texto` (texto, obligatorio)

- `autor` (texto)
 - `obra` (texto)
 - `anio` (entero, nullable)
 - `creado_en` (timestamp)
3. **Generar la migración desde ese esquema:** `diesel migration generate crear_citas --diff-schema`. Diesel compara `schema.rs` con la BD y escribe `up.sql` y `down.sql`. Es el `alembic revision --autogenerate`.
 4. Aplicar y revertir para comprobar: `diesel migration run`, `diesel migration revert` y `diesel migration redo`. Equivalen a `alembic upgrade` y `alembic downgrade`.
 5. Crear una segunda migración, `sembrar_citas`, con unos 10 INSERT de citas iniciales (Cien años de soledad, Anna Karenina...). Su `down.sql` las borra.
 6. **Migraciones embebidas:** añadir la `crate diesel_migrations` y usar `embed_migrations!("migrations")`. En `get_engine()`, después de conectar, ejecutar `run_pending_migrations`. Así la app instalada crea y actualiza su BD sola, sin la CLI (esto es clave para la Tarea 5).
 7. Registrar `mod schema;` en `main.rs`.

Conceptos de Rust que aparecen

- **Macros declarativas** (`table!`, `embed_migrations!`): generan código en tiempo de compilación. Conviene explicar qué expanden (se puede ver con `cargo expand`).
- **Módulos** (`mod`, `pub`, `use crate::...`).
- **Result y el operador ?** al manejar el resultado de las migraciones.

Criterio de aceptación

- Al borrar `divagando.db` y ejecutar `cargo run`, la BD se recrea con la tabla y las citas semilla.
 - `diesel migration redo` funciona sin errores.
-

Tarea 2 — Capa de datos: listar, obtener y guardar citas

Objetivo: funciones de acceso a datos que se puedan probar y que no dependan de la UI.

Pasos

1. **Separar librería y binario:** crear `src/lib.rs` que exponga `config`, `schema`, `models` y `repo`. `main.rs` pasa a usar `divagando::...`. Es necesario para las pruebas de integración de la Tarea 4.
2. **`src/models.rs`:**
 - `Cita`, para leer. Deriva `Queryable`, `Selectable` y `Debug`, y usa `#[diesel(table_name = ...)]`.
 - `NuevaCita`, para insertar. Deriva `Insertable`. Sus campos son `&str` con `lifetime 'a`, o `String` si se prefiere evitar `lifetimes` al principio.
3. **`src/repo.rs` (o `src/citas.rs`):** todas las funciones **reciben `&mut SqlConnection` como parámetro** y devuelven `QueryResult<...>`:
 - `listar_citas` devuelve todas las citas ordenadas por `id`.
 - `obtener_cita(id)` devuelve `Option<Cita>` usando `.find(id).first().optional()`.
 - `cita_aleatoria()` usa `ORDER BY RANDOM()` mediante `define_sql_function!`. Es la función principal de la app.
 - `guardar_cita(nueva)` devuelve la `Cita` creada usando `insert_into(...).returning(...)`.
4. `get_engine()` se queda solo como punto de entrada que usan `main` y la UI. Allí se obtiene la conexión con `.lock()` y se pasa `&mut *guard` al repositorio.
5. Probar a mano desde `main.rs`: imprimir la lista y una cita aleatoria.

Decisión de diseño

El repositorio **no** llama a `get_engine()` por dentro. Si lo hiciera, las pruebas no podrían usar una BD en memoria. El singleton se mantiene porque encaja bien con los callbacks de GTK, que exigen closures `'static`.

Conceptos a explicar cuando aparezcan

- `#[derive(...)]`: qué implementa por ti cada trait.
- **Lifetimes en NuevaCita<'a>**: por qué un struct que guarda `&str` necesita `'a`.
- **Option frente a Result**, y `.optional()` para convertir "no encontrado" en `None`.
- **MutexGuard y deref (&mut *guard)**: por qué hace falta el `*`.
- **Los tipos de Diesel**: son muy verbosos en los mensajes de error del compilador. Conviene aprender a leer un error de "trait bound not satisfied".

Criterio de aceptación

- `cargo run` lista las citas semilla, muestra una aleatoria, guarda una nueva y la recupera por id.
-

Tarea 3 — Interfaz GTK4 + Libadwaita con Blueprint

Objetivo: una ventana nativa que muestra una cita aleatoria, lista las citas y permite agregar una nueva.

Pasos

1. Dependencias:

- `gtk4` (renombrada a `gtk` en `Cargo.toml`) y `libadwaita` (renombrada a `adw`), con la feature de versión que

- corresponda a lo instalado.
 - `glib-build-tools` como build-dependency.
2. **Blueprint** en `ui/ventana.blp`:
- Una `Adw.ApplicationWindow` con `Adw.HeaderBar` y un `Adw.ViewStack` de 3 páginas:
 - **"Cita"**: una etiqueta grande con el texto y otra con autor y obra, más un botón "Otra cita".
 - **"Todas"**: un `Gtk.ListBox` dentro de un `ScrolledWindow`.
 - **"Nueva"**: `Adw.EntryRow` para texto, autor y obra, más un botón "Guardar".
 - Cada widget que el código necesite lleva un `id`.
3. **build.rs**:
- Invoca `blueprint-compiler batch-compile` para convertir `.blp` en `.ui` dentro de `OUT_DIR`.
 - Luego `glib_build_tools::compile_resources` empaqueta los `.ui` según `ui/resources.gresource.xml`.
 - Añadir cargo: `rerun-if-changed=ui`.
4. **src/ui.rs**: construir la ventana con `gtk::Builder::from_resource` y obtener widgets con `builder.object("id")`.
- Se elige esto **en lugar de CompositeTemplate y subclassing**. Esa alternativa exige `ObjectSubclass`, un módulo `imp`, `glib::wrapper!` y otras piezas, mucho boilerplate para aprender. Se puede migrar después como ejercicio avanzado.
5. **Conectar señales**:
- `button.connect_clicked(...)` llama al repositorio y actualiza las etiquetas.
 - "Guardar" valida que el texto no esté vacío, guarda, limpia el formulario y refresca la lista.
 - Los errores se muestran con un `adw::Toast`.
6. **main.rs**:
- ```
adw::Application::builder().application_id("dev.divaga")
 luego connect_activate(ui::construir) y app.run().
```

**Conceptos a explicar (la parte más verbosa del proyecto)**

- **Closures y move:** por qué los callbacks de GTK exigen `'static` y cómo se clonan widgets dentro del closure. `glib::clone!` y su sintaxis especial (`#[weak]`, `#[strong]`) son lo más confuso de esta tarea.
- **Conteo de referencias:** los widgets de GTK son baratos de clonar (un clon es otra referencia al mismo objeto). Comparar con Rc.
- **build.rs:** qué es un build script y cuándo se ejecuta.
- **Genéricos y turbofish:**  
`builder.object::<gtk::Label>("id").`

## Criterio de aceptación

- `cargo run` abre la ventana.
  - "Otra cita" cambia la cita.
  - Guardar una cita nueva la hace aparecer en "Todas" y en las aleatorias.
- 

## Tarea 4 — Pruebas unitarias y end-to-end

---

### Unitarias / integración de datos

1. Crear un helper de pruebas que abra `SqlConnection::establish(":memory:")` y ejecute las migraciones embebidas de la Tarea 1. Cada prueba recibe una BD limpia y aislada.
2. Pruebas en `tests/repo.rs`, contra la librería de `lib.rs`:
  - listar después de la semilla;
  - obtener un id existente y uno inexistente (`None`);
  - guardar y recuperar;
  - obtener una cita aleatoria cuando la tabla está vacía.
3. Pruebas unitarias pequeñas con `#[cfg(test)] mod tests` dentro del módulo, por ejemplo para la validación del formulario si se extrae como función pura.

## E2E (UI)

- GTK exige un solo hilo, así que las pruebas de UI van en `tests/ui.rs`, con `#[gtk::test]` o con `cargo test -- --test-threads=1`. Cada prueba:
  - construye la ventana con una BD en memoria;
  - simula clics con `button.emit_clicked()`;
  - comprueba el texto de las etiquetas y el número de filas de la lista.
- Para que esto funcione, `ui::construir` debe recibir la conexión o un "contexto" en vez de llamar a `get_engine()` directamente. Es un refactor pequeño que conviene prever en la Tarea 3.
- Opcional: añadir `cargo clippy` y `cargo fmt --check` como parte del "done".

## Conceptos

- **Atributos** `#[test]` y `#[cfg(test)]`.
- **La diferencia entre `src/` (tests unitarios) y `tests/` (tests de integración)** en Cargo.
- **Aserciones:** `assert_eq!`, `assert!` y `matches!`.

## Criterio de aceptación

- `cargo test` pasa en verde sin depender de `divagando.db` ni del `.env`.

---

## Tarea 5 — Instalador para Windows

---

### Pasos

1. Añadir `#[windows_subsystem = "windows"]` en `main.rs`

para que no se abra la consola, activo solo en release con `cfg_attr`.

## 2. Ubicación de la BD en producción:

- Si no hay `.env`, usar `glib::user_data_dir()`, que en Windows es `%LOCALAPPDATA%`. La ruta queda así: `%LOCALAPPDATA%\divagando\divagando.db`.
- Crear la carpeta con `std::fs::create_dir_all`.
- El `.env` queda solo para desarrollo.

## 3. Compilar con `cargo build --release`.

## 4. Empaquetar el runtime de GTK junto al `.exe`:

- Las DLLs de `C:\gtk-build\gtk\x64\release\bin` que use el ejecutable.
- `share\glib-2.0\schemas` (compilados).
- Los iconos de Adwaita.
- Los loaders de `gdk-pixbuf`.
- Documentar la lista en `docs/empaquetado.md`.

## 5. Instalador con Inno Setup (script `installer/`

`divagando.iss`):

- Copia la carpeta de distribución a `Program Files`.
- Crea un acceso directo en el menú Inicio.
- Incluye desinstalador.
- Alternativa si se prefiere un MSI: `cargo-wix`.

## 6. Icono de la app (`.ico`) incrustado en el `.exe` con la `crate winres` desde `build.rs`.

## Criterio de aceptación

En una máquina o usuario de Windows **sin** Rust ni GTK:

- se instala;
- abre la app;
- crea la BD en `%LOCALAPPDATA%` con las citas semilla;
- se desinstala limpio.

---

## Orden y dependencias

---

0 → 1 → 2 → 3 → 4 → 5

El helper de BD en memoria de la Tarea 4 puede adelantarse a la Tarea 2 para escribir pruebas mientras se desarrolla, y se recomienda.

## Archivos que se crearán o modificarán

---

- Cargo.toml
- build.rs
- diesel.toml
- .env
- migrations/\*
- src/main.rs
- src/lib.rs
- src/schema.rs
- src/models.rs
- src/repo.rs
- src/ui.rs
- src/config/db.rs: sumar migraciones embebidas y la ruta de datos del usuario.
- ui/ventana.blp
- ui/resources.gresource.xml
- tests/repo.rs
- tests/ui.rs
- installer/divagando.iss
- docs/\*.md

## Verificación global

---

- cargo build, cargo clippy, cargo test y cargo run al cerrar cada tarea.
  - Al final, instalar el .exe generado por Inno Setup en un usuario limpio y recorrer el flujo: ver cita, otra cita, listar, guardar y reiniciar la app para confirmar que la cita nueva sigue ahí.
-

## Siguiente paso tras aprobar

---

Guardar este requerimiento como `docs/requerimiento_desarrollo.md` en el repositorio.